

Precieuse AHOOMEY-ZUNU
Jules Hinson

Rapport de Projet – Labyrinthe

1. Introduction

1.1 Contexte

Le projet consiste à réaliser une version informatique du jeu de société Labyrinthe en Java.

L'objectif principal était de développer une application graphique permettant à plusieurs joueurs de déplacer leurs pions dans un labyrinthe dynamique afin de récupérer des trésors.

1.2 Objectifs

Les principaux objectifs étaient :

- mettre en œuvre une architecture logicielle propre ;
 - séparer l'interface graphique de la logique métier ;
 - appliquer les notions vues en cours (JavaFX, observables, graphes, sérialisation, persistance) ;
 - proposer une application extensible et maintenable.
-

2. Architecture du projet

2.1 Architecture MVVM

Le projet repose sur l'architecture MVVM (Model – View – ViewModel).

Model

Le modèle contient les règles du jeu et les structures de données principales :

- Game
- Maze
- Tile
- Player
- Treasure

Le modèle ne dépend pas de JavaFX.

View

La vue est développée avec JavaFX.

Elle est responsable de :

- l'affichage du plateau ;
- l'affichage des joueurs ;
- la gestion des interactions utilisateur.

Afin de simplifier l'interface et d'améliorer la réutilisabilité du code, certains composants graphiques ont été encapsulés dans des classes dédiées. Par exemple, la classe **AddPlayerButton** regroupe toute la logique liée à l'ajout de joueurs (joueur invité, profil enregistré ou bot) au sein d'un composant unique.

ViewModel

Les ViewModels assurent la liaison entre le modèle et l'interface graphique.

Ils exposent les données nécessaires à l'affichage et traduisent les actions utilisateur en appels au modèle.

3. Gestion des données observables

Afin de découpler le modèle de JavaFX, le projet utilise son propre système d'observables.

ObservableValue

Permet d'observer une valeur unique.

ObservableList

Permet d'observer une collection.

Adaptation vers JavaFX

La classe FxAdapter convertit les observables internes du projet vers les observables JavaFX.

Cela permet au modèle de rester indépendant du framework graphique.

4. Représentation du labyrinthe

4.1 Structure du labyrinthe

Le labyrinthe est représenté par une grille de tuiles.

Chaque tuile possède :

- une orientation ;
- des sorties ;
- éventuellement un trésor ;

- éventuellement une position de départ.

4.2 Construction du labyrinthe

La classe MazeBuilder est utilisée pour construire le plateau.

Elle :

- place les tuiles fixes ;
 - ajoute les tuiles libres ;
 - mélange les tuiles ;
 - génère la tuile supplémentaire utilisée pendant la partie.
-

5. Utilisation des graphes

Pour calculer les déplacements possibles, le labyrinthe est transformé en graphe.

MazeToGraphAdapter

Chaque tuile devient un nœud.

Deux tuiles sont reliées lorsqu'elles possèdent des sorties compatibles.

Paths

La classe Paths réalise un parcours en largeur (BFS).

Elle permet :

- de déterminer si une case est accessible ;
 - de reconstruire un chemin ;
 - d'obtenir les déplacements possibles d'un joueur.
-

6. Intelligence artificielle et simulation

GameSimulation

Le projet contient un moteur de simulation indépendant du modèle principal.

Cette classe permet :

- de tester des coups ;
- d'évaluer des déplacements ;
- d'annuler les modifications effectuées.

La simulation repose sur un mécanisme de rollback permettant d'explorer rapidement plusieurs possibilités sans modifier la partie réelle.

7. Sauvegarde et persistance

Sérialisation

L'état du jeu est converti en objets DTO puis sérialisé au format JSON.

Base de données SQLite

La classe `SqlPersistenceService` assure :

- la sauvegarde des parties ;
- le chargement de la dernière partie ;
- la gestion des profils utilisateurs.

Les données sont stockées localement dans une base SQLite.

8. Services

Le projet regroupe plusieurs services :

DialogService

Permet aux ViewModels de demander des interactions utilisateur sans dépendre directement de JavaFX.

Services

Classe centralisant l'ensemble des services utilisés par l'application.

Cette approche simplifie l'injection des dépendances.

9. Difficultés rencontrées

Au cours du projet, plusieurs difficultés ont été rencontrées :

- gestion des observables ;
- synchronisation entre modèle et interface graphique ;
- représentation du labyrinthe sous forme de graphe ;
- implémentation de la persistance ;
- conception du système de simulation.

Ces difficultés ont conduit à plusieurs refactorisations du projet.

9.1 Adaptation de l'affichage des cases

Au cours du développement, la classe **CellView** a été modifiée afin d'améliorer l'affichage du plateau. Initialement, elle héritait directement de **TileView**, ce qui limitait les possibilités d'affichage à une simple tuile.

Nous avons choisi de faire hériter **CellView** de **StackPane** afin de pouvoir superposer plusieurs éléments graphiques sur une même case. Cette modification permet d'afficher simultanément la tuile, le joueur présent sur la case ainsi que les informations associées aux trésors. Pour faciliter leur identification, les trésors sont représentés par les lettres de l'alphabet (A, B, C, ...), chaque lettre correspondant à un identifiant de trésor.

Cette approche permet également de gérer plus facilement les effets visuels et les interactions utilisateur tout en conservant une interface claire et lisible.

10. Conclusion

Ce projet a permis de mettre en pratique plusieurs concepts importants du développement logiciel :

- programmation orientée objet ;
- architecture MVVM ;
- JavaFX ;
- graphes ;
- observables ;
- sérialisation ;
- persistance des données.

Le résultat obtenu est une application modulaire, maintenable et facilement extensible.